RAG (Retrieval Augmented Generation)

1. INTRODUCCIÓN

RAG es una arquitectura que combina la búsqueda de información en una fuente externa con la capacidad de generación de un LLM (modelo grande de lenguaje). Su objetivo es permitir que un modelo genere respuestas utilizando información relevante recuperada de documentos, bases de conocimiento u otras fuentes de datos, en lugar de depender exclusivamente de lo aprendido durante su entrenamiento.

Como ya vimos en clases, un LLM de propósito general, podría explicar cómo suelen funcionar en instituciones educativas las reglas de convivencia. Sin embargo, esa respuesta no garantiza que conozca las reglas particulares de esa institución. Si el modelo responde utilizando únicamente su conocimiento previo, podría proporcionar un dato incorrecto con una redacción convincente.

Una solución a este problema, consiste en recuperar el fragmento del reglamento que contiene la normativa de asistencia y proporcionarles al modelo junto con la pregunta del estudiante. De esta manera, el modelo puede elaborar una respuesta basada en el contenido específico del reglamento.

Entonces, la idea central del RAG es **separar la recuperación de información de la generación de respuestas y conectarlas dentro de un mismo flujo de procesamiento.**

2. ¿Qué problema resuelve RAG?

Los modelos grandes de lenguaje presentan algunas limitaciones importantes cuando se utilizan como sistemas de consultas sobre información específica:

1. **Conocimiento limitado:** EL modelo no conoce automáticamente los documentos internos de una empresa, los apuntes de una materia o los procedimientos de una organización.
2. **Conocimiento potencialmente desactualizado:** El entrenamiento de un modelo se realiza con datos correspondientes a determinados periodos, esto quiere decir que, no podemos asumir que conoce las modificaciones de una normativa, un producto o un procedimiento.
3. **Limitaciones de Contexto:** Hoy en día, los modelos grandes de lenguaje tienen como límite de contexto hasta 1 millón de tokens, esto en

muchísimo a comparación de hasta hace 1 o 2 años, sin embargo, tenemos que tener en cuenta que, si necesitamos alimentar a nuestro modelo con un PDF de 200 páginas, por cada pregunta que hagamos, esas mismas 200 páginas irán una y otra vez en el historial de mensajes, lo que podría ocasionar que las primeras preguntas sean respondidas correctamente, pero en la pregunta 5, 6 o 7, el modelo empiece a inventar información, a lo que llamamos **alucinaciones**.

4. **Alucinaciones:** Ya vimos cómo funciona por debajo un modelo de tipo decoder-only. La generación es autorregresiva: en cada paso el modelo produce una distribución de probabilidad sobre el siguiente token y elige uno (muchas veces el más probable). Pero que un token sea el más probable según los patrones aprendidos en el entrenamiento **no garantiza que la respuesta sea verdadera.** El modelo optimiza la plausibilidad del texto, no su veracidad. Además, como la respuesta suele ser gramaticalmente impecable, resulta muy convincente aunque sea incorrecta.

La arquitectura RAG permite abordar estos problemas recuperando información relevante en el momento de la consulta. No obstante, es importante recalcar que, en sí el RAG **no elimina automáticamente las alucinaciones, ni garantiza que siempre una respuesta va a ser correcta**. La calidad de la respuesta va a depender, entre otras cosas, de la calidad de los **documentos**, de la **recuperación realizada** y de **la capacidad del modelo para utilizar adecuadamente la información proporcionada.**

3. Diferencias entre RAG, Fine Tuning y Prompt Engineering

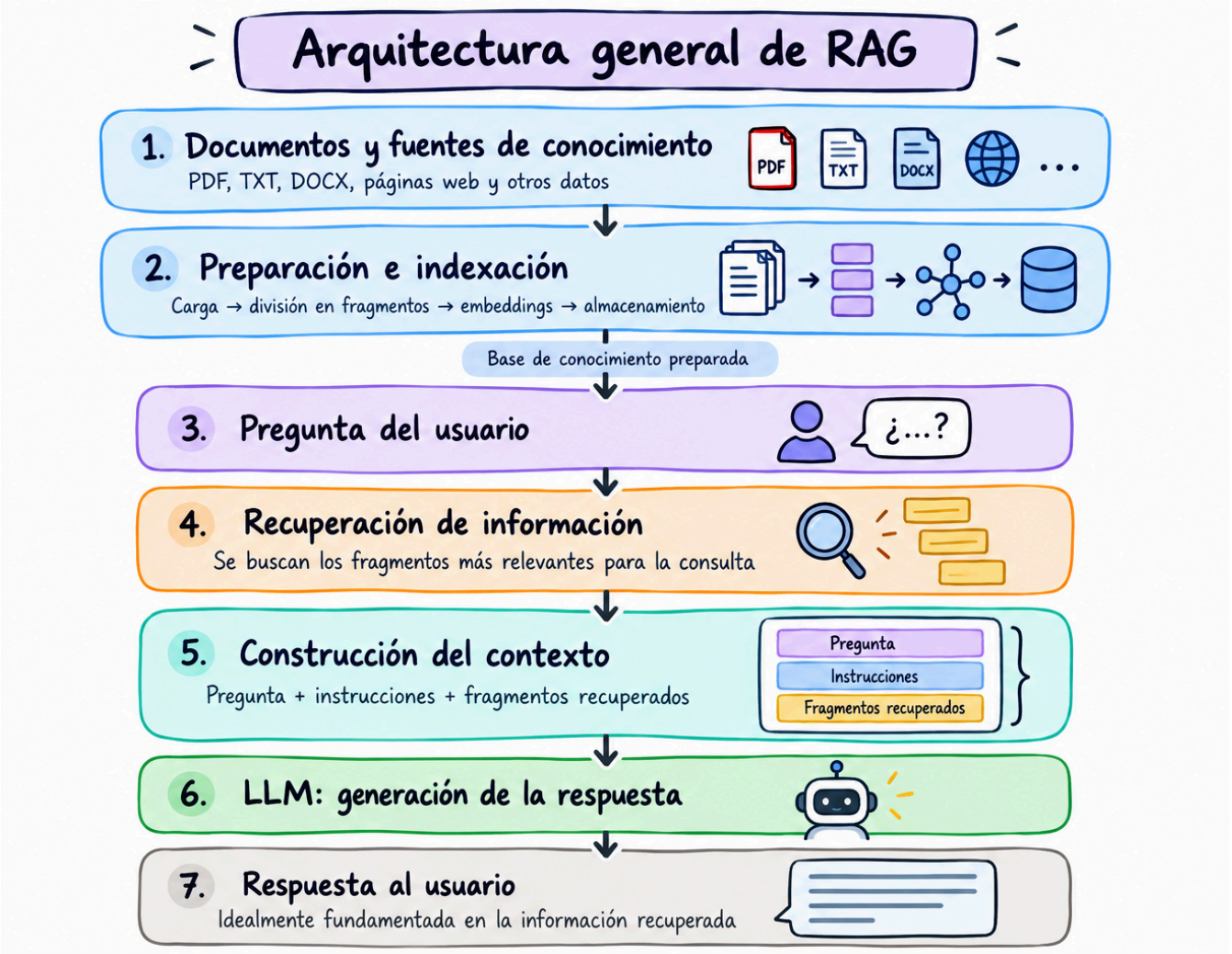
Hay que diferenciar bien estos 3 conceptos. El fine tuning es un proceso de tomar un modelo de inteligencia artificial o de aprendizaje automático que ya ha sido entrenado de manera general, y volverlo a entrenar con datos específicos y propios para que se especialice en una tarea, un tono o un sector en concreto. El Rag en cambio, agrega conocimiento sin reentrenar el modelo, ni cambiar parámetros de entrenamientos. Por otra parte, cuando hablamos de Prompt Engineering o Ingeniería de instrucciones, consiste en técnicas para estructurar y optimizar las preguntas o comandos que se le envían a un modelo de inteligencia artificial generativa. PD: esto obviamente es pregunta del siguiente examen.

4. Arquitectura general de RAG

Una arquitectura RAG puede dividirse en dos procesos principales:

1. **Indexación:** Es el proceso de preparar los documentos para que puedan buscarlos eficientemente.
2. **Consulta o recuperación y generación**: Buscar información relevante para una pregunta y utilizarla para producir una respuesta.

Esta separación es muy importante porque los documentos se suelen preparar antes de que un usuario realice la consulta.

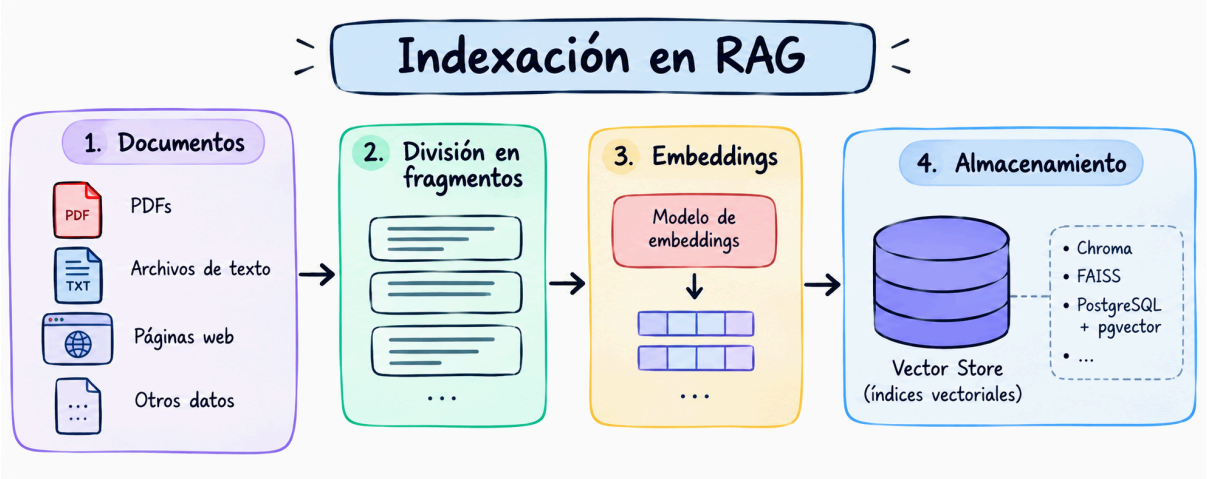


4.1 Proceso de Indexación

La indexación consiste en preparar la información que estará disponible para las consultas futuras. En términos generales, el proceso incluye los siguientes pasos:

1. Obtener los documentos desde las fuentes correspondientes.
2. Extraer su contenido textual.
3. Dividir el texto en fragmentos que sean manejables y entendibles.
4. Convertir esos fragmentos en vectores numéricos, mediante un modelo de **embeddings.**
5. Almacenar los vectores junto con el texto que representa y sus metadatos en una estructura que permita recuperarlos.

El resultado sería una base de conocimiento para realizar búsquedas y obtener documentos que se parezcan **semánticamente**. Obviamente, no es necesario realizar este proceso cada vez que un usuario hace una consulta. En una aplicación real, normalmente se indexan los documentos previamente y se actualizan cuando se incorporan, modifican o eliminan fuentes.

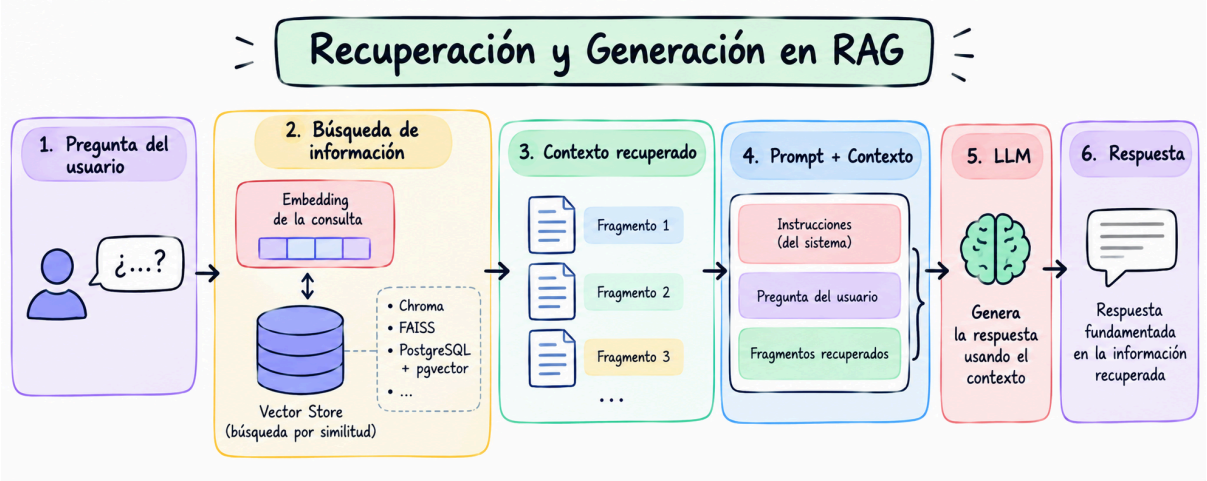


4.2 Proceso de Consulta: Cuando un usuario realiza una pregunta, el sistema ejecuta otro flujo:

1. Recibe la consulta.
2. La consulta se convierte en un embedding.
3. Se realiza una búsqueda semántica para encontrar los fragmentos más relevantes del índice.
4. Recupera el texto original asociado con esos fragmentos.

5. Envía una solicitud sobre el modelo de lenguaje.
6. Devuelve una respuesta generada.

La recuperación vectorial es una técnica muy utilizada, pero no es la única opción. También es posible combinar búsquedas por palabras claves, filtrar por metadatos y mecanismos híbridos.



5. Componentes fundamentales de RAG

5.1. Documentos y fuentes de conocimiento

Los documentos constituyen la fuente de información que el sistema utilizará para responder consultas. Esta información puede estar en archivos PDFs, DOCXs, Bases de Datos, etc. Un buen sistema debe estar preparado para recibir archivos en varios formatos.

La fuente debe ser adecuada para el problema que queremos resolver. Recordemos que, nuestro LLM, utilizará las fuentes que nosotros le proveemos para responder las consultas del usuario. Si la información es incorrecta, está desactualizada o es ambigua, la recuperación no puede garantizar una respuesta confiable.

También es importante conservar la información sobre el archivo, como el número de página, el título, o la sección donde pertenece. Estos datos se los conoce como metadatos y pueden utilizarse también para filtrar resultados, mostrar referencias o rastrear la procedencia de una respuesta.

5.2. Carga de documentos (document loader)

Un Document Loader es un componente que permite leer información desde una fuente y convertirla a una estructura que podamos procesar dentro de una aplicación.

En Langchain, muchos cargadores representan el contenido mediante objetos Document. Un objeto de este tipo tiene principalmente:

- page_content: el texto del documento o fragmento.
- metadata: información adicional sobre su procedencia u otras características.

Por ejemplo, al cargar un documento PDF, un sistema podría producir un objeto con el texto de una página y metadatos que indiquen el nombre del archivo y el número de página. Lo que nos permite trabajar con diferentes formatos mediante una misma interfaz, sin tener que diseñar desde cero un proceso de extracción para cada fuente. Sin embargo, cargar un documento no significa que ya esté listo automáticamente, sino que tenemos que realizar cierto proceso de limpieza de contenido, dividir el documento en fragmentos y generar sus embeddings.

5.3. División de texto (Text Splitting o Chunking)

Los documentos extensos suelen dividirse en unidades más pequeñas llamadas chunks o fragmentos. Supongamos que disponemos de un manual técnico de 200 páginas, si tratamos todo el manual como una única unidad, podemos tener dificultades para recuperar solamente la sección relevante para una pregunta. En cambio, si lo dividimos en fragmentos, podemos localizar partes específicas del contenido y proporcionar al modelo solamente la información necesaria.

Existen diferentes estrategias de división:

- **División por caracteres:** La más simple, separa el texto según la cantidad determinada de caracteres.
- **División por tokens:** Utiliza unidades de tokenización para controlar mejor el tamaño del contenido.
- **División recursiva:** Intenta dividir primero por separadores naturales, como párrafos o saltos de línea, y recurre a separadores más pequeños cuando es necesario.
- **División semántica:** Busca identificar cambios de significado o de tema para establecer los límites de fragmentos.

Tamaño de los fragmentos

EL tamaño de los fragmentos es una decisión importante de diseño. Si los fragmentos son demasiados pequeños, podemos perder información necesaria para interpretar una idea. Por ejemplo, una definición podría quedar separada de sus condiciones o excepciones. Si son demasiado grandes, pueden incluir información irrelevante, dificultar la recuperación precisa y consumir más espacio en el contexto del modelo. Como todo en IA, no existe un tamaño universalmente óptimo, depende del tipo de documento, de las consultas separadas, del modelo de embeddings y del modelo generativo. Se suele configurar al principio, un tamaño máximo de 4000 caracteres.

Solapamiento entre fragmento

Es habitual permitir cierto solapamiento entre fragmentos consecutivos, esto significa que, una parte del texto incluido al final de un fragmento también aparece al comienzo del siguiente. Por ejemplo: si dividieramos un texto en fragmentos de 500 caracteres con un solapamiento de 100, los fragmentos consecutivos compartirán parte de su contenido.

El solapamiento ayuda a evitar que una idea importante quede dividida justo en el límite de dos fragmentos. Sin embargo, un solapamiento excesivo aumenta el volumen de información almacenada y puede producir resultados redundantes.

5.4. Embeddings

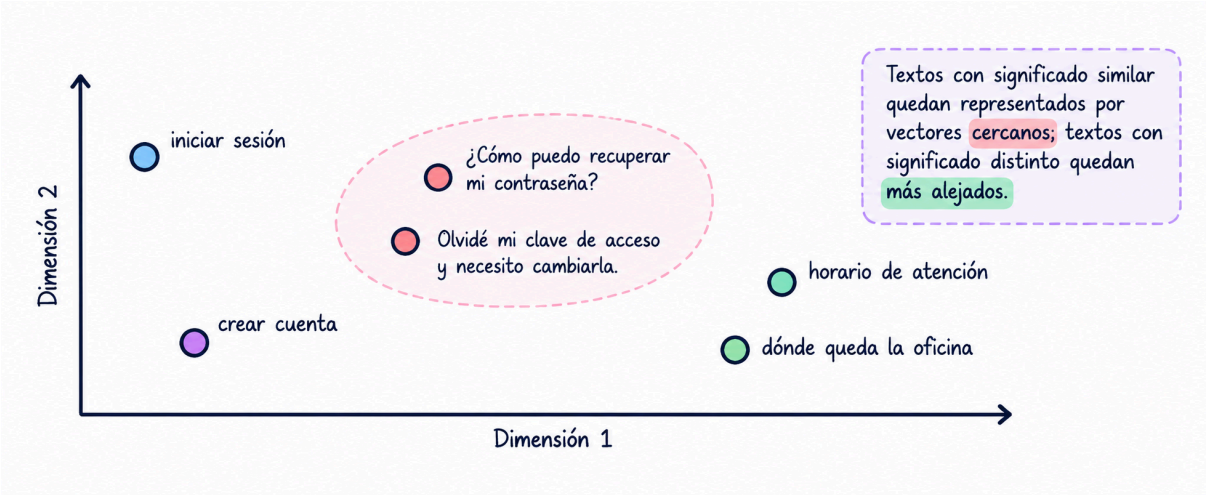
Un embedding es una representación numérica de un elemento, como una palabra, una oración o un fragmento de texto, dentro de un espacio vectorial.

Un modelo de embedding transforma el texto en un vector de números que captura determinadas características semánticas y lingüísticas. Por ejemplo, los siguientes textos expresan ideas relacionadas:

- «¿Cómo puedo recuperar mi contraseña?»
- «Olvidé mi clave de acceso y necesito cambiarla».

Aunque utilicen palabras diferentes, un buen modelo de embeddings puede representarlos de manera tal que sus vectores resulten cercanos en el espacio vectorial.

Para entender un poco mejor, veamos la siguiente ilustración:



Es importante destacar que un embedding no es un resumen, una traducción ni una respuesta a una pregunta, sino que es una representación numérica de las palabras que nos facilita hacer operaciones con ellas, entre una de esas operaciones, tenemos la comparación semántica.

¿Cómo se representan los textos?

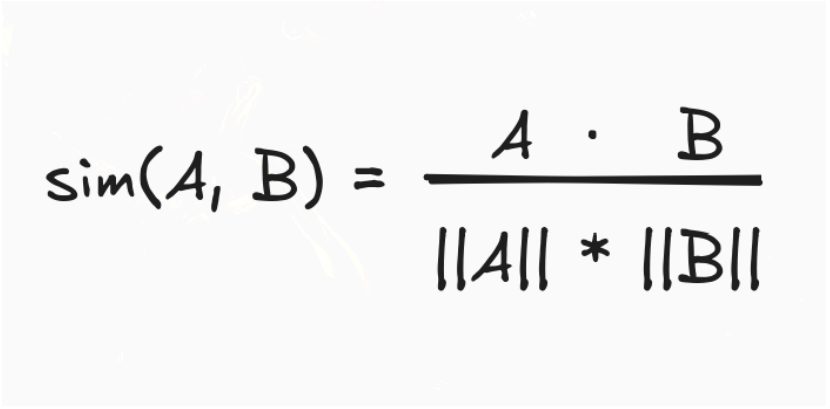
Imaginemos que un modelo de embedding, representa cada fragmento mediante un vector de cuatro dimensiones, solamente para fines ilustrativos:

$$v = [0.12, -0.48, 0.73, 0.21]$$

En aplicaciones reales, los vectores suelen tener muchas más dimensiones, dependiendo obviamente del modelo utilizado. Cada fragmento queda asociado con su presentación vectorial. Durante la consulta, el modelo genera otro vector a partir de input ingresado, y ese vector es comparado con los otros vectores que tenemos almacenados en una base de datos especial, una **base de datos vectorial.**

¿Cómo se mide la similitud?

Una de las medidas más utilizadas es la similitud del coseno. Esta compara la orientación de dos vectores y permite estimar qué tan relacionados están según la representación aprendida.-



Donde A y B son vectores, $A \cdot B$ es el producto escalar y las normas representan las magnitudes de los vectores.

Hay que tener en cuenta que, no todas las bases de datos vectoriales realizan la misma operación. Siempre hay que verificar que tipo de estrategia utilizan para medir la similitud.

5.5. Bases de datos vectoriales

Una base de datos vectorial permite almacenar representaciones vectoriales y recuperar elementos relacionados con un vector de consulta. En una aplicación RAG, se suele almacenar o asociar:

- Un embedding de cada fragmento.
- El texto original.
- Los metadatos correspondientes.
- Un identificador que permite localizar ese elemento.

Cuando llega una consulta, el sistema busca los vectores más relevantes y devuelve los fragmentos asociados. Existen diferentes alternativas tecnológicas, como ChromaDB, Qdrant, Pinecone y otras soluciones que incorporan capacidades vectoriales. También existen bases de datos relaciones que ofrecen extensiones para almacenar y consultar vectores, como PostgreSQL con su extensión pgvector.

5.6 Retriever (Recuperador)

El recuperador es el componente responsable de recuperar documentos relevantes a partir de una consulta. En una búsqueda vectorial típica, el sistema genera un embedding para la pregunta, compara ese vector con los almacenados y devuelve los fragmentos seleccionados.
En langchain, por ejemplo, tenemos que configurar la cantidad de **k-resultados** para relevantes, donde k representa la cantidad de fragmentos a retornar. Por ejemplo, con k=4, el sistema intentará recuperar cuatro fragmentos relevantes siempre que existan suficientes resultados y la implementación los devuelva.
Hay que tener en cuenta que, al elegir un valor de k muy bajo, recuperaremos muy pocos fragmentos, lo que podría ocasionar que omitamos información necesaria. Elegir un valor de k muy alto, podría introducir ruido, redundancia y consumo innecesario de tokens.

5.7. Modelo de Lenguaje (LLM)

El LLM será el componente que genera la respuesta en lenguaje natural. Una vez recuperada la información, la aplicación construye un prompt que incluye las instrucciones, los fragmentos relevantes y la pregunta del usuario. El modelo utiliza el contexto para elaborar una respuesta. Por ejemplo, podríamos darle la siguiente instrucción:

```python
prompt = f"""
Responde la pregunta utilizando únicamente la información proporcionada en: {contexto}.
Si el contexto no contiene información suficiente para responder, indicá que no hay información suficiente.
"""
```

La variable **contexto** hará referencia a los documentos obtenidos en la búsqueda semántica.

6. Implementación de un RAG utilizando Langchain

Langchain es un marco de trabajo (framework) de código abierto, diseñado para facilitar la creación de aplicaciones basadas en LLMs. Está disponible tanto para Python como para JavaScript y actúa como puente que conecta a la IA con datos externos y herramientas del mundo real.

Documentación oficial: https://www.langchain.com/

Para esta práctica, nos apoyaremos en Ollama para la utilización de un LLM y del modelo de embedding. Langchain cuenta con una integración con Ollama, llamada langchain-ollama.

1. Creamos un entorno virtual con uv y lo montamos

```shell
uv venv env && source env/bin/activate # En Linux
```

o

```shell
uv venv env && .\env\bin\activate # En windows
```

2. Instalamos las siguientes dependencias:

```shell
uv pip install langchain langchain-ollama langchain-chroma
langchain-community langchain-text-splitters pypdf marimo
```

Algo a tener en cuenta es que, langchain está dividido en varios paquetes de Python, en lugar de incluir todas sus funcionalidades dentro de una única biblioteca.

Paquete	Descripción
langchain	Componentes de alto nivel y abstracciones para construir aplicaciones con LLM.
langchain-core	Interfaces y componentes fundamentales: mensajes, prompts, ejecución mediante Runnable, etc.
langchain-community	Integraciones comunitarias con diferentes fuentes de datos y herramientas.
langchain-text-splitters	Herramientas para dividir textos en fragmentos.
langchain-ollama	Integración con Ollama para utilizar modelos de lenguaje y embeddings.
langchain-chroma	Integración con Chroma como base de datos vectorial.

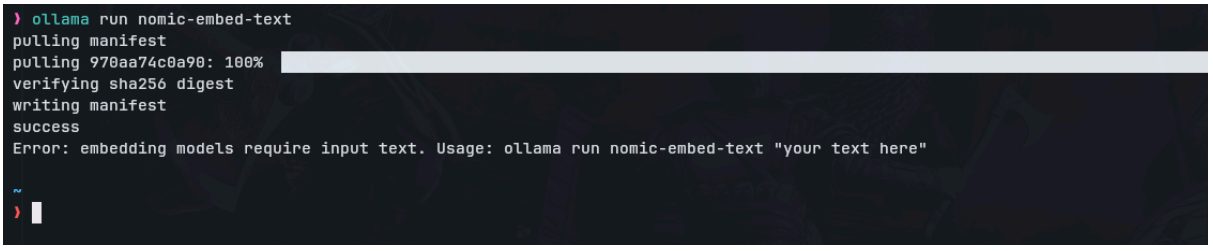
3. Modelo de Embedding: Vamos a descargar un modelo de embedding con ollama, llamado **nomic-embed-text**. Es un modelo open source, desarrollado por openai. Utiliza por defecto 768 dimensiones. Ahora necesitamos ejecutar el siguiente comando para descargarlo:

```shell
ollama pull nomic-embed-text
```

Una vez finalizado la descarga, si ejecutamos lo siguiente:

```shell
ollama run nomic-embed-text
```

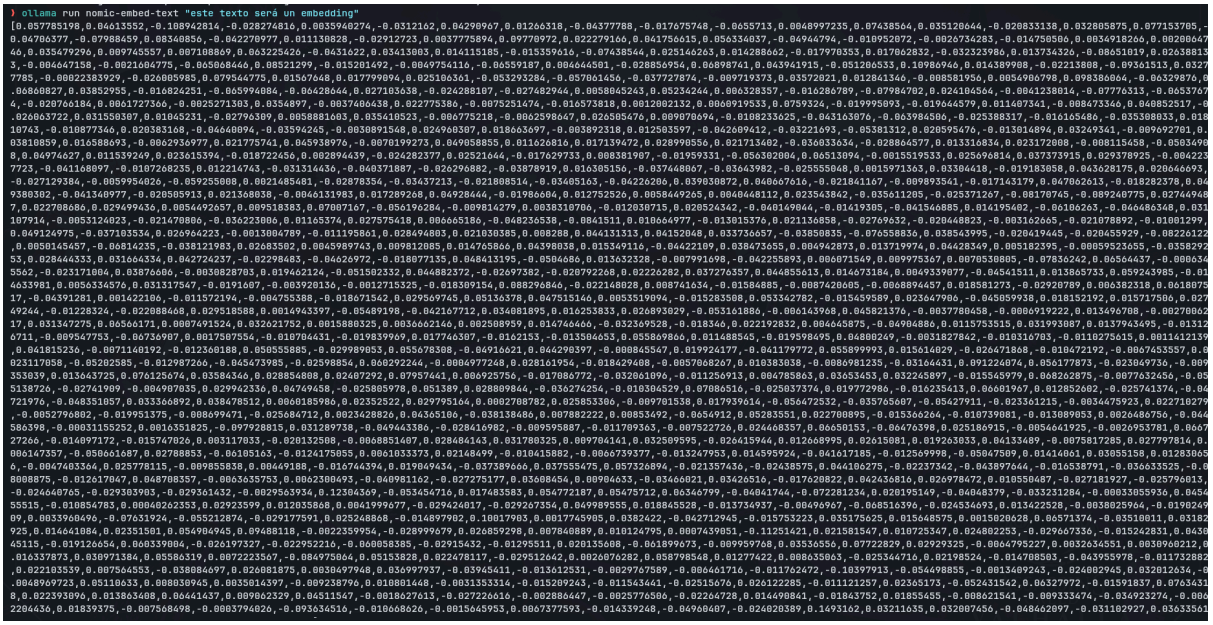
Nos saldrá el siguiente error:



Y es que, no podemos usar este modelo como los que veníamos usando en la materia. Pero, podemos usarlo de la siguiente forma:

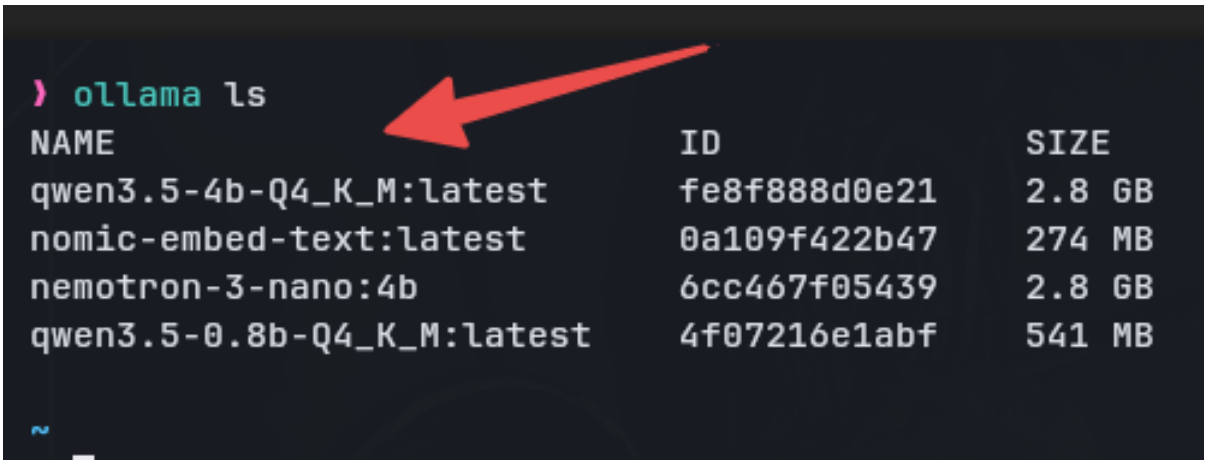
```shell
ollama run nomic-embed-text "este texto será un embedding"
```

Y obtendremos un resultado como este:



Un arreglo con 768 elementos, la representación numérica de "`este texto será un embedding`"

Para el modelo encargado de responder las consultas, utilizaremos el LLM que cuantizamos en clases anteriores:



Si no tienen el modelo, pueden utilizar cualquier otro o **hacer la tarea** para tener el modelo**.**

Indexación de documentos en la base de datos vectorial

1. Carga de documentos: Langchain ofrece distintas clases para cargar documentos. En este caso, vamos a probar con un PDF elaborado por los profes de esta materia: https://aprende.formosa.gob.ar/api/lessons/2ffa3bfc-adb5-4f57-9991-8fe275548a40/media/637b240a-79ce-49f7-af23-af2a44b01e42?download

Como se trata de un PDF, vamos a utilizar **PyPDFLoader**

```python
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("Fundamentos de REDES NEURONALES.pdf") # cargamos el pdf

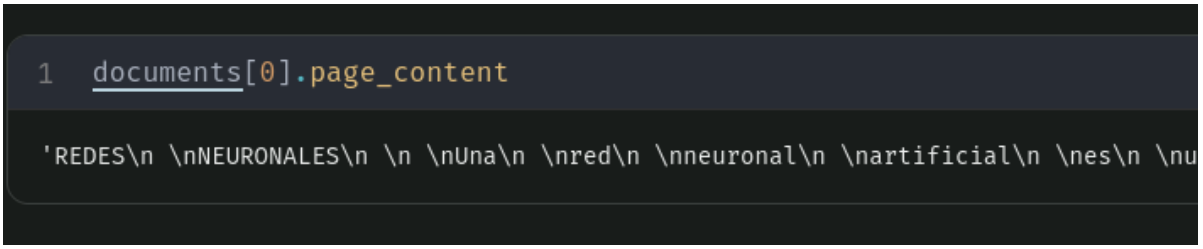
documents = loader.load() # convertimos el pdf en una lista de objetos Document
```

El resultado de load() es una lista de objetos Document. En nuestro caso, al cargar un único archivo, tendremos normalmente un documento cuyo contenido está disponible en page_content y cuyos metadatos incluyen información sobre la fuente.

Podemos inspeccionar el contenido de la siguiente forma:

```python
documents[0].page_content
```

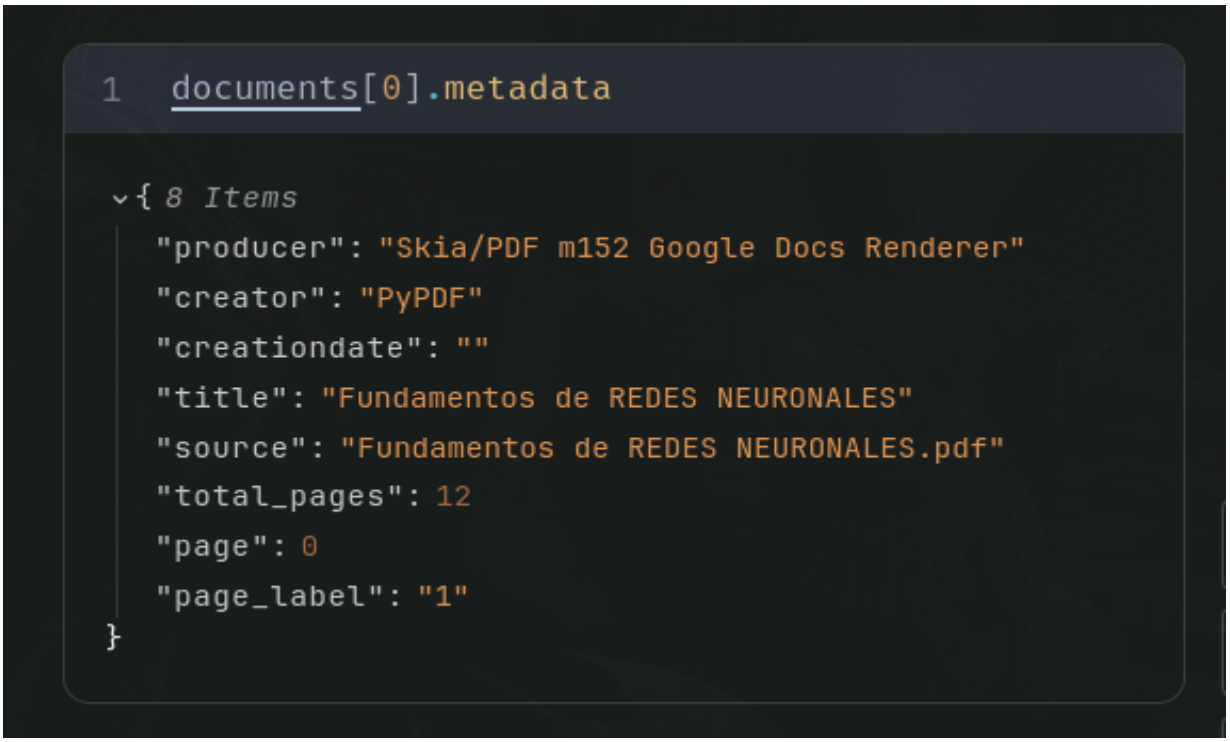
Resultado:



Y a los metadatos:

```python
documents[0].page_content
```

Resultado:



2. Dividir el documentos en fragmentos

Ahora vamos a dividir el contenido en fragmentos más pequeños. Langchain nos provee de herramientas que nos van a permitir elegir una estrategia de chunking y declarar el tamaño max de cada fragmento y su overlap (pequeño contexto del chunk anterior)

```python
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)

chunks = text_splitter.split_documents(documents)
print("Cantidad de chunks: ", len(chunks))
```

En este caso, estamos declarando un límite máximo de 1000 caracteres, con 200 caracteres como máximo de solapamiento.

3. Crea la función de embeddings

```python
from langchain_ollama import OllamaEmbeddings

embeddings= OllamaEmbeddings(
    model="nomic-embed-text"
)
```

Con eso ya preparamos el modelo de embedding, necesario para la creación de nuestra instancia de chromadb.

4. Construir la base de datos vectorial

Langchain tiene una integración con Chroma, llamada **langchain_chroma.** Con esta integración, podemos crear una conexión a chromadb para nuestros embeddings, declarando lo siguente:

- embedding_function: El modelo de embedding a utilizar.
- collection_name= Las colecciones en Chroma, podemos verlas como una especie de tablas, donde vamos a guardar los embeddings que hagamos. Algo a tener en cuenta es que, lo ideal sería, crear una colección por cada tema que vamos a manejar. En este caso, crearemos uno solo para practicar.

```python
from langchain_chroma import Chroma

vector_store = Chroma(
    embedding_function=embeddings,
    collection_name="redes_neuronales",
    persist_directory="./vector_store"
)
```

5. Creamos y guardamos los embeddings en chroma

Utilizando el metodo add_documents, guardamos los embeddings en nuestra base de datos.

```python
vector_store.add_documents(chunks)
```

Cabe aclarar que este método es **síncrono y bloqueante**: el código que lo invoca no continúa hasta que la operación termina (cálculo de embeddings y escritura en el vector store). En una aplicación web esto importa: si lo llamamos directamente dentro de un endpoint async (por ejemplo en FastAPI), bloqueamos el event loop y el resto de las solicitudes quedan esperando hasta que finalice.

Por eso, existe otro método llamado aadd_documents. aadd_documents es la versión asíncrona de add_documents. Esa "a" inicial es una convención de LangChain: casi todos sus métodos tienen un gemelo con "a" adelante (invoke → ainvoke, embed_documents → aembed_documents, similarity_search → asimilarity_search).

```python
await vector_store.aadd_documents(chunks)
```

Cuando el código llega a un **await**, la función le dice al event loop: "esto va a tardar (por ejemplo, esperando la respuesta de Ollama), mientras tanto atiende otras tareas". Así, un mismo proceso puede manejar muchos usuarios a la vez sin quedarse parado esperando.

6. Creación del Retriever

Necesitamos una función que nos permita retornar los documentos mas relevantes según el input del usuario. Vamos a crear una función que reciba como parámetro una query y la cantidad de k.

```python
def retriever(query: str, k=5) -> list:
    results = vector_store.similarity_search(query, k=k)
    return results
```

similarity_search es el método del vector store que busca los fragmentos más parecidos a una consulta. Es la operación central de la fase de consulta en RAG.

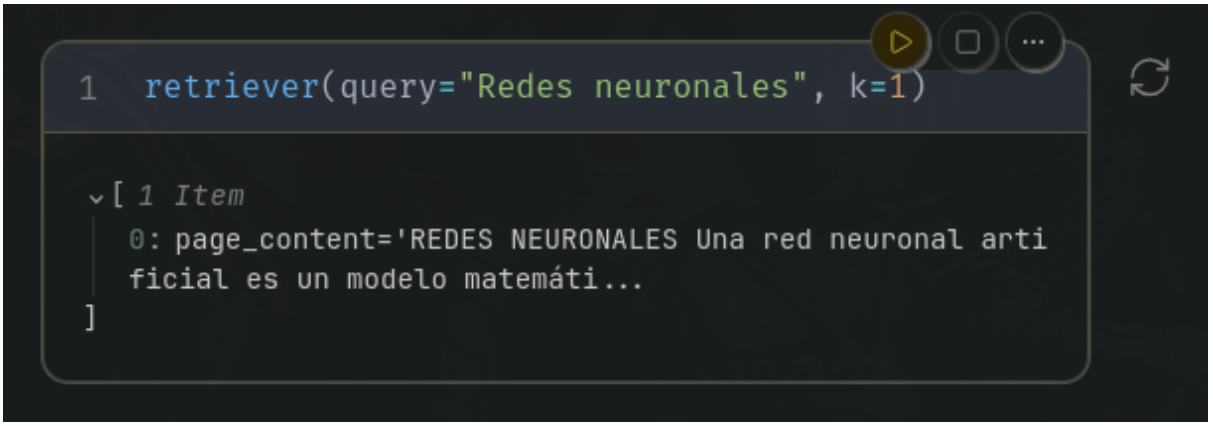
Qué hace, paso a paso:

1. Recibe un texto (la pregunta del usuario).
2. Calcula el embedding de ese texto, usando el mismo modelo de embeddings con el que indexamos.
3. Compara ese vector contra los vectores guardados, midiendo la distancia (coseno, L2, etc., según la configuración del vector store).
4. Devuelve los k documentos más cercanos, como una lista de Document.

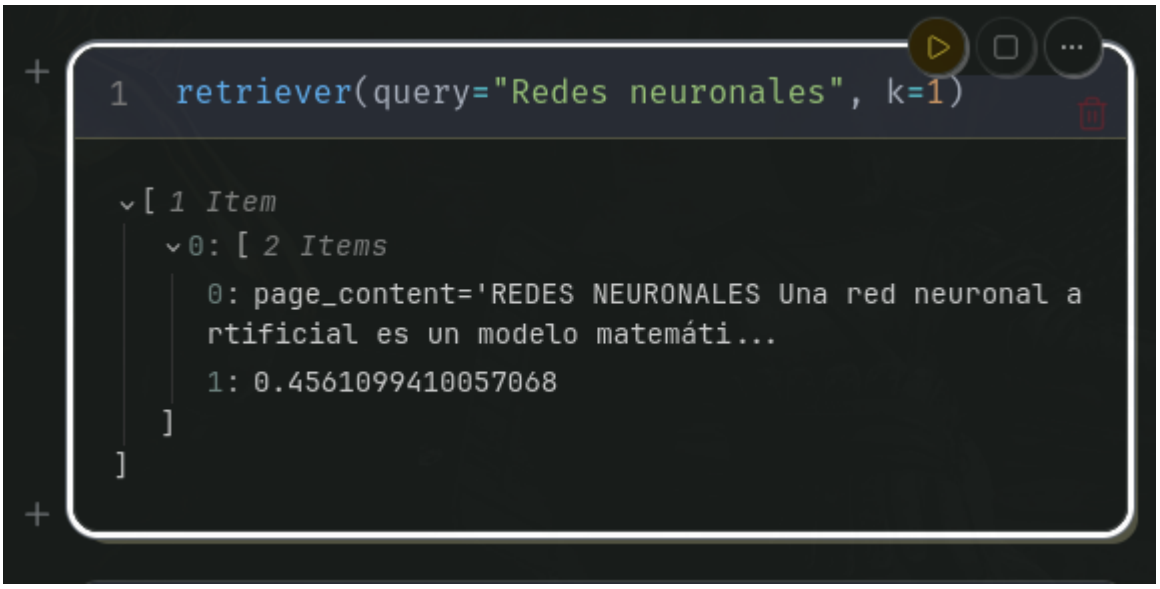
Veamos un ejemplo:

```python
retriever(query="Redes neuronales", k=2)
```

Resultado:



También existe el método similarity_search_with_score(). A diferencia del anterior este nos devuelve el puntaje de cada resultado, osea que tan parecidos son los embeddings o que tan cerca está en el espacio multidimensional.

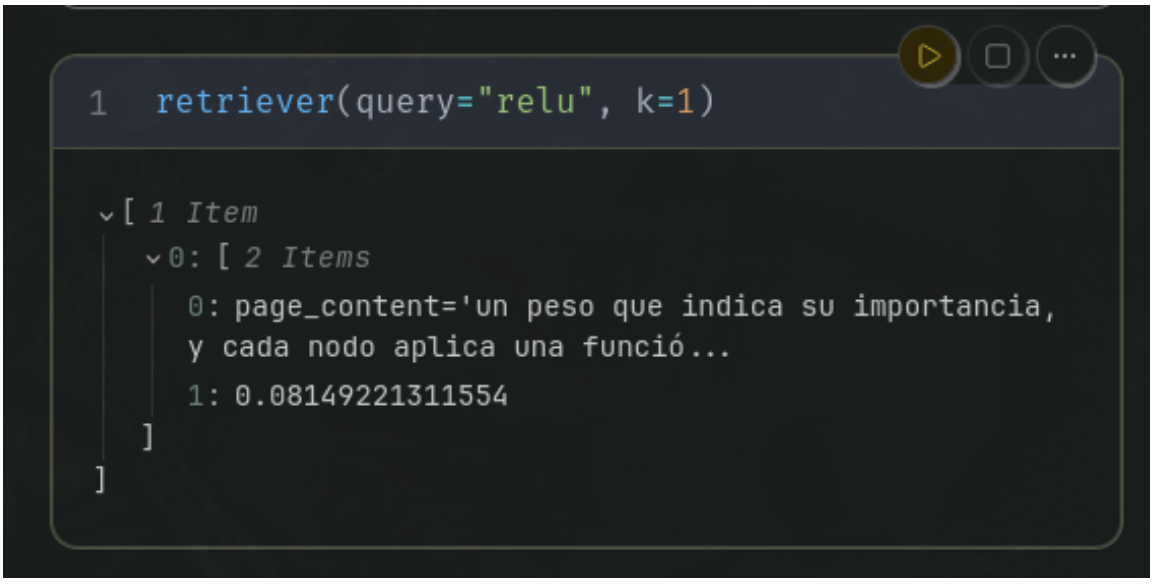


Cuando más cercano a 0 está, más similares son.

Otro metodo util es **similarity_search_with_relevance_scores.** Como lo dice su nombre, solo nos devuelve los fragmentos con score "relevante" e ignora al resto.

```python
retriever(query="Relu", k=1)
```

Resultado:



7. Crear el prompt template

Con la herramienta **ChatPromptTemplate** de langchain, vamos a definir la estructura de nuestro prompt:

```python
from langchain_core.prompts import ChatPromptTemplate


prompt = ChatPromptTemplate.from_template(
    """
    Eres un asistente especializado en responder preguntas academicas.
    Responde a las preguntas utilizando siempre el contexto que se te provea.
    Si el contexto no tiene información suficiente, responde solo "No tengo Información suficiente para responder."

    Contexto: {context}
    Pregunta: {question}
    """
)
```

8. Generar la respuesta:

El último componente es el LLM. Vamos a utilizar OllamaLLM para comunicarnos con el modelo generativo.

```python
from langchain_ollama import OllamaLLM

model = OllamaLLM(
    model="qwen3.5-4b-Q4_K_M",
    reasoning=False, # configuramos en False por el problema de qwen 3.5
temperature=0
)
query = "Que son las redes neuronales"
context = retriver(query= query, k=10)
model.invoke(prompt.invoke({
    "context": context,
    "question": query
}))
```

Resultado:

```
Basado en el documento proporcionado ("Fundamentos de REDES NEURONALES"), aquí tienes una explicación de qué son las redes neuronal

**Definición y Funcionamiento Básico**
Las redes neuronales son sistemas inspirados en el funcionamiento del cerebro humano, compuestos por unidades básicas llamadas **ne

**Componentes Clave de una Neurona:**
Según el ejemplo del documento, cada neurona funciona mediante los siguientes pasos:

1.  **Entradas ($X$):** Reciben datos de entrada. En el ejemplo, se usan variables como $X_1$ (Horas de estudio) y $X_2$ (Clases as
2.  **Pesos ($W$):** Son valores numéricos que determinan la importancia de cada entrada. Se obtienen durante el proceso de entrena
3.  **Bias ($b$):** Es un valor de desplazamiento que permite ajustar la salida de la neurona. En el ejemplo: $b = -2$.
4.  **Cálculo de la suma ponderada ($z$):** La neurona calcula una suma combinando las entradas con sus pesos y el bias:
    $$z = (W_1 \cdot X_1) + (W_2 \cdot X_2) + \dots + (W_n \cdot X_n) + b$$
    *En el ejemplo: $z = (0.3 \cdot 5) + (0.6 \cdot 8) - 2 = 4.3$.*
5.  **Función de Activación ($\phi$):** Es la función que transforma el valor $z$ en la salida final de la neurona. Le da "personal
    *   **Ejemplo de Sigmoid:** En el caso del ejemplo, al aplicar la función sigmoide a $z=4.3$, se obtiene un resultado de aproxi
    *   **Ejemplo de ReLU (Rectified Linear Unit):** Es otra función muy utilizada. Si el valor es negativo, lo "apaga" (lo convier

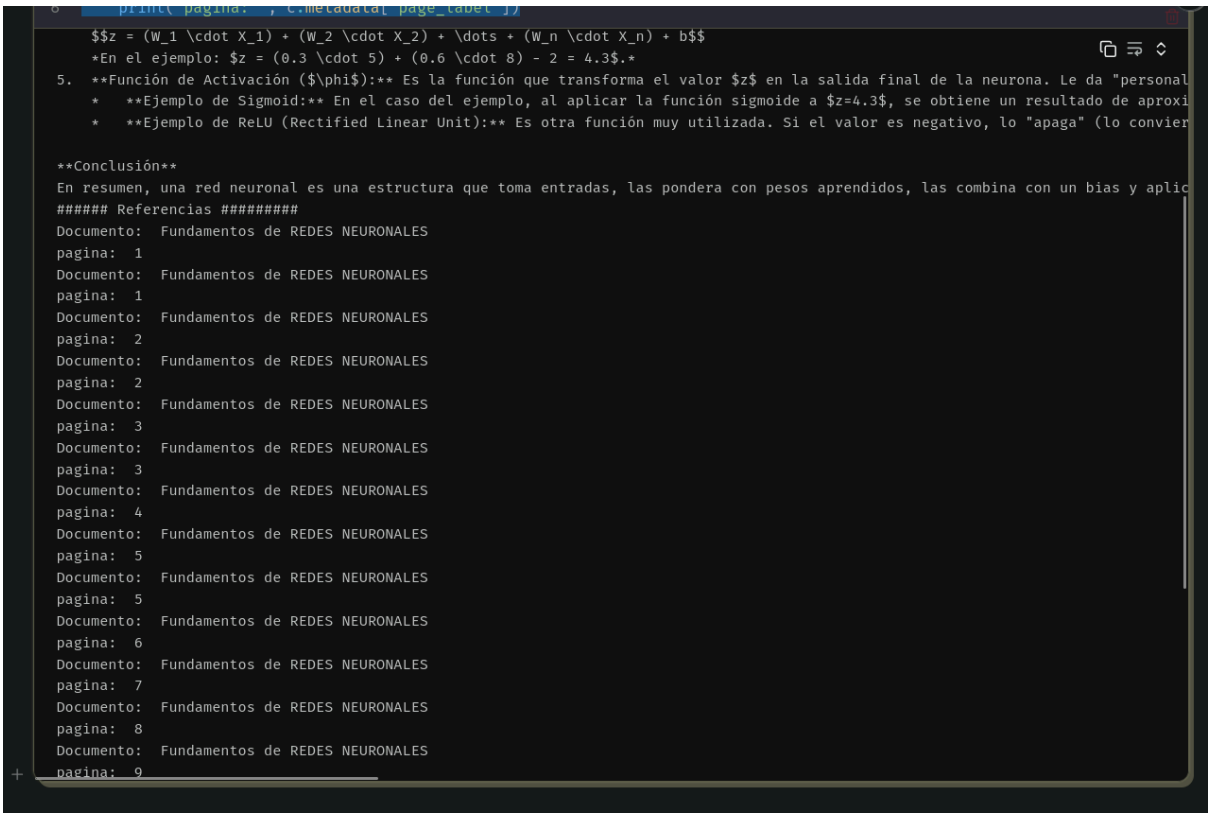
**Conclusión**
En resumen, una red neuronal es una estructura que toma entradas, las pondera con pesos aprendidos, las combina con un bias y aplic
```

Algo muy importante al configurar nuestro modelo es la **temperatura**.
La Temperatura es un hiperparámetro que controla el nivel de aleatoriedad y creatividad de un LLM. Cuando más alta sea la temperatura, más creativo tiende a ser nuestro modelo, pero tenemos un riesgo más alto de alucinación. Cuando más baja es la temperatura, nuestro modelo dará respuestas más “deterministas”, osea que casi siempre elegirá la opción con mayor probabilidad estadística.

Y si quisiéramos ver las referencias, podemos hacer lo siguiente:

Python
```python
print("###### Referencias #########")
for c in chunks:
    print("Documento: ", c.metadata["title"])
    print("pagina: ", c.metadata["page_label"])
```

Resultado:



7. Conclusión

La arquitectura RAG (Retrieval-Augmented Generation) representa una solución efectiva para mitigar las limitaciones de conocimiento y alucinaciones en los modelos de lenguaje, permitiendo conectar fuentes de datos privadas o actualizadas en tiempo real sin requerir costosos reentrenamientos. Al combinar el poder de recuperación semántica con la capacidad generativa de un LLM, se obtienen respuestas precisas, contextualizadas y fundamentadas en evidencia concreta.

Asimismo, la utilización de modelos cuantizados (como las versiones Q4_K_M) para la etapa de generación en un esquema RAG resulta ser una estrategia óptima en entornos con recursos computacionales limitados. Al reducir la precisión de los pesos del modelo, se logra un consumo de memoria RAM significativamente menor y una mayor velocidad de inferencia en hardware local (como Ollama), manteniendo al mismo tiempo una precisión suficiente para sintetizar correctamente la información recuperada del contexto.